

Sybil-Based Attacks on Google Maps or How to Forge the Image of City Life

Cihan Eryonucu
eryonucu@kth.se

KTH Royal Institute of Technology
Stockholm, Sweden

Panos Papadimitratos
papadim@kth.se

KTH Royal Institute of Technology
Stockholm, Sweden

ABSTRACT

Location-based services (LBS) increasingly rely on participatory or crowd-sensed data: users voluntarily contribute data about their whereabouts and points of interest (POIs) and allow the LBS to capture the dynamically changing environment, e.g., how crowded specific places, streets, or public transportation are. Popular LBS applications do not offer strong security, less so for their participatory sensing (PS) and data contribution part. Openness favors participation and increases data, but it also makes attacks easier. Sporadic misbehavior incidents and the presumed user honesty should not be reassuring: an attacker could exploit the PS components and submit a large volume of forged data to dominate the PS-collected LBS data, locally or at a large scale. Individuals, organizations, or entire areas could be targeted, e.g., having customers diverted or causing public transportation routes or roads to appear congested. The lingering open question is whether such attacks can be perpetrated against well-established popular LBS with PS components. This paper affirms this: we investigate Google Maps, the single most popular application in this domain, and show a range of effective and scalable attacks based on very modest adversarial assumptions. We reverse-engineer the data submission process and automate attacks that craft and submit false data in volume and a targeted fashion. We collect evidence that our attacks work on POI crowdedness, traffic congestion levels, and public transportation crowdedness with extreme caution. We responsibly disclosed the attacks to Google, acknowledged them and awarded recognition. The attack methodology carries over to other LBS applications but, most importantly, raises awareness and motivates countermeasures, which we also outline here, for stronger LBS and PS security overall.

CCS CONCEPTS

• **Security and privacy** → *Software reverse engineering; Software security engineering.*

KEYWORDS

Participatory Sensing, Mobile Crowd Sensing, Sybil Attacks, Google Maps, Attacks, Security, Vulnerabilities

ACM Reference Format:

Cihan Eryonucu and Panos Papadimitratos. 2022. Sybil-Based Attacks on Google Maps or How to Forge the Image of City Life. In *Proceedings of*

the 15th ACM Conference on Security and Privacy in Wireless and Mobile Networks (WiSec '22), May 16–19, 2022, San Antonio, TX, USA. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3507657.3528538>

1 INTRODUCTION

Location-based services (LBS) provide rich data to users, covering a broad gamut of points of interests (POIs), e.g., regarding landmarks, public services, businesses, as well as navigation and trip planning, including public transportation. Significant parts of this information, e.g., road layout, location of a shopping center, or a tram route, are static or change slowly. In addition, dynamically changing information, e.g., traffic conditions, numbers of shoppers, or tram passengers, is made available to LBS users. Often, such data can be collected effectively only thanks to the contributions of users, who consent to share their location or submit specific information about specific POIs; or even submit time- and geo-stamped sensory measurements (e.g., noise, light) their devices are equipped to collect. Such participatory data collection, termed in general participatory sensing (PS) or mobile crowd sensing (MCS), greatly enhances the user experience for numerous LBS, with growing popularity. In fact, the broader the user participation is, the richer and more diverse the participatory sensed data are; thus, the more successful the LBS is with a broader user base, e.g., Google Maps, Apple Maps, Waze.

Overall, participating users are presumed honest, contributing accurate data, typically automatically through a client application they download, install and configure, but cannot, in principle, modify. Data is collected in compliance with the user agreement, user preferences, and possibly specific data collection campaigns they agree to join. Submitted data can sometimes be faulty due to benign failures, e.g., Global Positioning System (GPS) coordinate inaccuracies in urban environments, sensing limitations and inaccuracies, or occasional careless user inputs. However, such irregularities do not drastically change the overall PS formed knowledge.

Most, if not all, PS applications are open to participation, they lack rigid defense mechanisms and they are far from offering strong security (and privacy); despite proposals in the research literature for secure and privacy-preserving PS architectures and LBS, e.g., [1, 2, 3]. Currently, widely adopted applications typically require creating a password-protected account, using an email address and a mobile phone, logging in, and submitting data. This has been the predominant model for mobile applications.

PS-based LBS depend on large user numbers: the less strict security policies and the less cumbersome security mechanisms are, the more usable and likely to gain in popularity such apps are. At the same time, there have been only isolated incidents of lesser consequence or research-driven investigations of attacks. For example, sporadic faulty Google Maps reports on adverse road conditions [4]



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike International 4.0 License.

WiSec '22, May 16–19, 2022, San Antonio, TX, USA.

© 2022 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9216-7/22/05.

<https://doi.org/10.1145/3507657.3528538>

were manually removed and ceased. Or, Simon Weckert, who carried 99 phones at walking speed in a cart [5], created the illusion of traffic congestion reported by Google Maps for streets he walked along. Or Ghost Riders, virtual vehicles created by an attacker appearing to move together to report faulty events effectively (e.g., traffic congestion) for a popular crowd-sourced navigation application [6].

Given the growing reliance of users on LBS with PS components, the open question is *whether such well-established popular LBS applications can be easily attacked and information manipulated in a targeted manner and in scale*. Investigating how to exploit PS data collection of different types, it is paramount to concretize risks and implications for currently deployed, popular applications. This is why we consider Google Maps, one of the most popular, versatile, and useful apps, thanks to its accuracy and dynamically changing information, thanks to millions of users contributing with their data [7]. Its PS-ed data collection is implicit (automatically collected user location) and explicit (user-based responses or submissions). Google Maps provides features ranging from detailed POI crowdedness information to public transportation [8].

We start by reverse-engineering the Google Maps PS-ed data submission process because it is not an open-source app, without a description of the involved protocols available. We seek to understand the types of data the app collects, how users participate, how often they do so, and whether submitted data can be replicated. Based on this understanding, we find exploits based on user registration, data submission, and user authentication. These weaknesses can lead to attacks that poison the PS functionality of Google Maps. We consider attacks of different types, based on the attacker capabilities and methodology.

Our first attack type, *script-based attacks*, create fake users, Sybils, using Python scripts, instead of real devices, mimicking the real app behavior. Sybils appear as if they travel in an area the same way normal Google Maps users would and change POI crowdedness levels in a coordinated manner. If the place appears crowded at a specific time, customers could decide to pick another, less crowded shop. We augment the attacker capabilities through *emulator-based attacks*, using smartphone emulators to launch multiple instances of the Google Maps app. Emulators are much cheaper and more configurable than actual devices, and they offer more attack options compared to script-based attacks. They can affect public transportation (e.g., a bus or tram), road traffic and place crowdedness levels. Attacking public transportation crowdedness can mislead users and create imbalances and worse passenger service, e.g., when users pick inappropriately. Attacking traffic levels can affect route selection by any driver using Google Maps for navigation, targeting individuals or causing traffic jams.

An attacker can use our approach here to harm other apps. Of course, each application deploys a different set of security measures and requires in-depth investigation separately. Still, lessons learned from circumventing existing security, e.g., OAuth2, TLS, two-factor authentication, can carry over to other PS-based LBS; along with the creation and orchestration of multiple accounts; a focused, automated, and large volume of data submission; and use of emulated devices for more refined control of the attacks.

The rest of the paper is organized as follows: Sec. 2 provides an overview of the Google Maps features and their data collection process. Sec. 3 explains the fundamental attack aspects and preliminaries. Sec. 4 gives details about reverse engineering the Google Maps. We outline the attacks in Sec. 5, and 6 respectively. We discuss the implications, and possible countermeasures in Sec. 7, with related work in Sec. 8. We explain the responsible disclosure and responsible conduct for this work in Sec. 9, before we conclude.

2 A GOOGLE MAPS PRIMER

Google Maps has various PS features. Here, we describe those we consider in our attack scenarios. *Places* or *points of interest* (POIs) in Google Maps refer to a broad number of interactable objects, such as restaurants, museums, hotels, businesses, schools, parks, etc, with detailed information when selected on the map.

Some *places*, e.g., businesses, and subway stations, show a histogram of average crowdedness level (in other words, popular times) of the place across the day [9]. There is no clear definition of what 'popularity' corresponds to; possibly, it relates to users visiting the place. For some places, popularity is shown in real-time, although not clear how much data or how many users over which period are necessary for the histogram creation and live update.

Google launched a new feature in 2019 for large cities [8], predicting how crowded public transportation (bus, subway, tram) lines are based on the historical crowdsourced data for any particular line. Google collects such data explicitly when it infers users took a line, based on their location, movement, or query by sending them a form asking how crowded the line was. Users can either ignore the form or respond with the appropriate crowdedness level as they perceive it. Google Maps creates popular times graphs, with hourly crowdedness levels, and shows a prediction per route/line.

Google Maps shows both historical and live road traffic levels based on contributions from the smartphones of drivers and passengers on the road [10]. When getting directions, Google Maps shows the busy or jammed parts of the route and presents alternative routes if the optimal route has some congested parts. Usually, location data is contributed if users log in with their Google accounts and give consent. There is a simple, unified registration process for all services, without, usually, auxiliary data (email or phone number). Users select permissions when they first launch the app to quickly reach the services. Google Maps is not an open-source application and there is no information about how the PS data is collected, explicitly or implicitly. Transport Layer Security (TLS) [11] is used for all communication between the server and the app, preventing eavesdropping and messages modification. After logging in with the app, the server issues authentication tokens which are used in the data submission process.

Place Crowdedness: The popular times graph is based on user location data, averaging the popularity over time [12]. Precise information on how much data a place needs, how many users need to contribute, and how long it takes to create the graph is not available. Typical app usage would be a user with an account traveling in the targeted area while the app runs in the background, sending data to the server, for creating the popular times graph of the target place. **Public Transportation:** Users contribute to the public transportation knowledge by responding to questionnaires *crowdedness forms*

when it is detected that a user used a specific transport line, based on her location. When a user queries directions to a destination, including public transportation, the app reviews possible lines along the route. The user does not need to start the route for the app to start the reviewing process. By checking the user location in real-time, Google Maps infers whether the user used (or is using) the specific line. When, for example, the user gets off the bus and unlocks her phone, the app sends the form, asking how crowded the bus was. The app downloads the bus route overview estimating the current bus location.

Road Traffic Congestion: To contribute to the aggregation of data used for live traffic congestion levels, users need to have an active driving route to some destination. The vehicle speed is the basis for estimating the road congestion levels. The app calculates the speed by using the GPS coordinates of the device. Based on the user/app's current location, the app informs on the congestion level of relevant road segments.

3 ATTACKING GOOGLE MAPS

The attacks target Google Maps PS-based features and specific relevant places/POIs. This can be a street or a sequence of streets, a route, whose perceived traffic level the attacker wishes to manipulate; or a restaurant whose crowdedness graph the attacker wishes to alter; or a specific bus line. Thus, the target depends on the motivation of the attacker and it can vary. We instantiate our attacks, without loss of generality, assuming the attacker focuses on single feature at a time. This is for presentation purposes, as an adversary could attack multiple features simultaneously.

We consider POI, public transportation, and traffic attacks. The first target is graphs for popular/crowded business times. The attacker wishes to make a place appear more popular and crowded than it is. The second target is public transportation line crowdedness: the attacker submits false data to make specific routes more crowded. The third target is live traffic congestion levels: the attacker emulates many slow-moving vehicles to make the targeted street(s) appear (relatively) congested.

Instead of a costly (and ineffective if in small scale) deployment of actual devices and colluding misbehaving users, the adversary would mount the attack(s) from, ideally, a single or a small number of computing platforms, either smartphones or main-stream workstations; targeting potentially any location or area, independently of the adversary whereabouts. To be successful, the adversary needs to appear as numerous Google Maps users, owning multiple distinct Google accounts; the higher the number, the more likely the attack success and the higher its effect. It may be infeasible for an attacker to control many accounts manually and simultaneously to create some effect. For more effective attacks, it is important for the attacker to automate and orchestrate the attacks, as well as scale up their volume. This can be done with the use of existing tools, Python and bash scripts, smartphone emulators, and auto-clicker apps. Scaling up based on the emulators is simple: once the emulator is configured, the attacker can replicate emulator images. **User Accounts:** At registration, a phone number is needed sometimes to verify the registering user. There are no clear rules but rather several observations [13, 14]. According to our experiments, the first five account creations from the same IP address do not

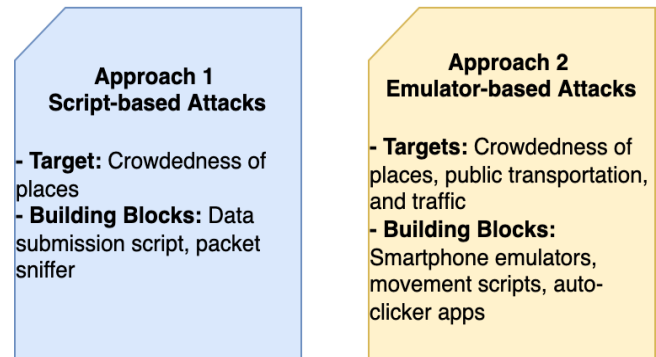


Figure 1: Overview of the Sybil-based attack approaches and targeted PS-based features.

require a phone number or an additional email address. After that, Google needs a phone number to send a verification text message. Users do not have to store the phone number while creating the account. They can simply leave the phone number field empty at the last step. One can verify up to five accounts with a single phone number but this can make those accounts trivially linkable. In total, an attacker can create five accounts with no connection to each other. The attacker would either need a not previously used IP address or an unused phone number for more accounts. To achieve this, one could use a VPN service or use free-of-charge cellular SIM cards (sometimes distributed freely in public places, e.g., subway entrances to lure new customers) that allow text message reception or attackers could buy/rent virtual phone services to receive SMS messages to bypass the phone verification [15]. Attackers could also directly buy Google accounts from third-party sites [16] with Google accounts on-sale typically created more than a year ago, so that they can submit reviews to Google Play Store and Google Maps. In principle, accounts that cannot be linked could be desirable from the attacker's viewpoint.

We created 20 accounts for our experiments, and they were created manually, using the browsers of our computers and smartphones. Five of them were created using one SIM card for verification codes. In this investigation, account creation is orthogonal and relevant only for large-scale attacks. We do not dwell on how to obtain many and we refrain from doing so. We did not buy any accounts from any vendor, nor did we use any virtual phone services to receive text messages, nor did we attempt any automated approach for account creation. Instead, we essentially created accounts the way actual, benign users are expected to do so.

Bypassing TLS: Google Maps is not an open-source app, thus to understand how user data/feedback is collected necessitates capturing communication between the app and the server, essentially reverse-engineering functionality. We investigate the application programming interface (API) calls between the app (client) and the server. Google Maps clients communicate with servers over TLS, thus, we cannot see the content of the API calls. We transform our computer into a man-in-the-middle (*mitm*) node to bypass the TLS. This is possible because the clients do not need to be authenticated but only the server does. All smartphone communication goes to

the *mitm* node, which then forwards data to the Google server; the server responses to the smartphone go through our *mitm* node too. We use Charles Proxy [17] to build this *mitm*-like packet capture capability. To bypass TLS, we create a self-signed certificate for the *mitm* node and install it to the smartphone, misleading it to initiate a TLS handshake with the *mitm* in lieu of the Google Maps server. iOS and Android (version 6 and below) operating systems allow users to trust third-party certificates, hence allowing us to complete the bypass process. With this scheme, we can see the Google Maps API calls and interpret how data collection and authentication work. This process is not an attack per se but a setup that allows us to do the necessary reverse-engineering investigation. The Google Maps server-only authentication allows us to bypass TLS to remedy this installing client certificates to the app is not straightforward, especially due to compatibility issues [18, 19].

We explore two **types of attack**, we term **script-**, and **emulator-based**, depending on the attack approach and tool. Script-based place attacks use scripts to automate the attack with minimum cost and human involvement. However, not every attack can be mounted this way. This is why we use smartphone emulators to carry out a broader gamut of attacks on places/POIs, public transportation lines and live road traffic. These are mostly automated and scalable attacks, which is essential from the attacker viewpoint.

Despite differences in the targeted features and attack tools, there are also similarities across attack approaches, as shown in Fig. 1. For clarity, we group script-based place attacks and emulator-based place attacks as *place attacks*, because they aim at submitting location data to places in Google Maps. We also group place, public transportation and traffic attacks, under Approach 2, *emulator-based attacks*, as they leverage smartphone emulators as an integral part of the attack(s). Table 1 shows the attack approaches, in columns, vs. features in rows, summarizing effectiveness and scalability.

The attacks are fundamentally simple: Google Maps is fed with user whereabouts. The goal of *place attacks*, submitting location data for specific POIs, and to create fake popularity and crowdedness. Such attacks can affect Google Maps users by creating wrong, misleading information, influencing user choices (e.g., visiting or not visiting a place) and thus their daily lives. Public transportation attacks submit crowdedness forms to create fake knowledge about specific public transportation lines. Similarly, traffic attacks creates fake traffic congestion levels. As a result, people may decide to use non-optimal routes for their intended destinations.

Initially, we learn from the system in Sec. 4. Then, in Sec. 5 and 6, we proceed with the precise steps taken by the adversary to compromise the data submission process, we present, respectively the effects of the proposed attacks we experimented with. We also briefly gauge their efficiency, even though we did not seek in this work to optimize in that sense and go beyond establishing feasibility for each attack, carefully crafting experiments with no risk for actual users.

4 REVERSE ENGINEERING GOOGLE MAPS

Having the setup to bypass TLS, we capture traffic between the iOS and Android versions of Google Maps and the server. Google uses OAuth2 [20] authorization tokens for authentication. The client attaches tokens to the custom HTTPS header for every API call.

	Script-based	Emulator-based
Place Crowdedness	✓✓	✓
Public Transportation	-	✓✓
Traffic Congestion	-	✓

Table 1: Effectiveness and evaluation of attack approach to Google Maps PS features. A double-check indicates an effective and scalable attack. A single check indicates a doable but moderately efficient attack (at this stage).

We identified three token types: *refresh*, *access*, and *submit* tokens. Refresh and access tokens are used by both the iOS and Android app versions; Android version 6 has no submit tokens. Below, we explain the use of each token type, all Base64 strings of 240-300 characters.

Refresh Tokens can be considered as long-term credentials, issued by the server when the user logs in with an account to the app. Refresh tokens have no expiration time and can be used until they are revoked; they are used only to obtain access tokens. Any user with a refresh token, can obtain all necessary authentication credentials, that is, other tokens, from the server. If the user decides to sign out from her account on the device, the app initiates a process of sending the refresh token to the server with a revocation request.

Access Tokens can be obtained from the server using the refresh token or another access token. There is no limit to obtaining access tokens: requesting one does not revoke the previously issued access token(s). The app sends an access token request, including device and account-specific information. For iOS, it sends the *client_id* the app registers per device when the user signs in and obtains a refresh token. If the app sends a different *client_id*, the server does not issue an access token. Thus, we can safely conclude Google Maps binds the refresh token with the registered *client_id*. For Android, the app sends more information to the server compared to the iOS version. Account-specific information, such as *email*, device-specific information, such as *android_id*, and software-specific identifiers, such as the *sdk* version. We found that Google does not verify most of these fields and still issues tokens even if field values, e.g., *sdk*, are incorrect. However, some essential fields need to be generated carefully. The *Service* field specifies the scope of the requested token; tokens are of no use outside their scope. For example, tokens requested for reporting location data cannot be used for user information retrieval. There are other essential fields, such as *client_sig*, *app_field*, and *token_request_option* but their values is same across all requests for user; hence, replaying captured packet values is suffices to get a fresh token.

Google Maps attaches the refresh token to the authorization field of the HTTPS header. Consequently, the server returns an access token with a specified expiration time. The token expiration time varies 50 to 85 seconds. Access tokens are used for obtaining more access tokens, submit tokens, and account information related to Google Maps. Android devices use the tokens for submitting data, but iOS devices use access tokens to work as intermediate authentication tokens. iOS access tokens have no scopes and are used for obtaining submit tokens.

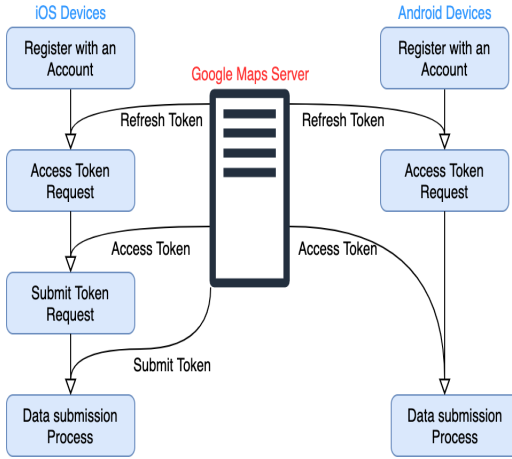


Figure 2: Overview of authorization and token acquisition.

Submit Tokens are short-term tokens used by iOS devices for reporting location data to the server. Submit tokens are obtained by sending a request to the server along with an access token. The client needs to define the scope for the requested submit token. Tokens become unusable after their expiration time, always with a value of 3599 ms.

App/client security actions: Having clarified the token functionality, we next explain the overall Google Maps client operation, starting with the user opening the app for the first time. We simplify the process, describing only authentication and application security actions. The user opens the app and logs in to her Google account via her username and password. Upon successful verification, the server returns the refresh token. In this step, the iOS Google Maps registers the *client_id* to the server, binding the *client_id* with the refresh token. Similarly, for Android devices, *client_sig* is registered. From this point on, iOS and Android apps differ in terms of the submission process. The iOS app obtains an access token using the refresh token. The apps use Access tokens mainly, while refresh tokens are used when there are no active tokens. The app obtains the submit tokens as soon as possible after obtaining an access token and it starts submitting data to the server. For Android devices, there is no submit token. Instead, the app requests access tokens to submit data. Both iOS and Android apps continuously renew their tokens. In our attacks, we follow the same steps the app takes.

Understanding the structure of data sent to the server: Every second, by default, the mobile client sends location data to the Google Maps server. These periodically sent packets have distinct features compared to other packets. *x-geo*, a custom HTTP header encoded as a BASE64 string, is particularly important. Sensed location data is not evident in the packets, but *x-geo* header might be the field carrying location information. Our first guess was decoding the *x-geo* to well-known strings, e.g., ASCII or Unicode. This did not yield any results. However, we found information on the usage of the *x-geo* in Google browser project Chromium’s source code [21]. There are code comments referred to usages and construction of the *x-geo* header. Furthermore, comments refer to the documentation of the *x-geo* header to an internal Google page [22], but one

needs Google internal access to see the contents. Still the source code gives good insights about the *x-geo* header: latitude, longitude, timestamp, and radius information is merged into a Protocol Buffer (protobuf) [23] object and then encoded as a BASE64 string. The protobuf file used in this code also exists in the chromium source code [24], which allows one to replicate this object. We reverse-engineer the encoding scheme to see whether the object in [24] is used in the *x-geo* headers used by Google Maps. Decoded *x-geo* headers fitted perfectly to the object in [24]. Extracted timestamp and location information matched the place and time of the captured packets. This established that *x-geo* headers carry information about the sensed location data. Using the same class, we can craft *x-geo* headers on demand.

The final aspect to look for is the content, the payload, of the submitted data. The app sends binary data in the payload, not encoded with an unknown encoding scheme. However, it appears parts of the payload can be decoded, revealing device-specific information (e.g., the device name and the OS version). The server accepts the replayed packets. We randomly change bytes in the payload and send the corrupted payload to see the effect. Sometimes the server does not accept the submitted packet. We use different payloads to investigate this, change every byte iteratively to find which bytes are essential for the server to accept the packets. 78% of the bytes, ‘safe bytes’, can be random values, while the rest must stay fixed to make the payload valid.

5 SCRIPT-BASED PLACE ATTACKS

These attacks aim to inflict maximum damage to the system with minimal human involvement. We create a Python script to simulate attacks with all the information via our reverse-engineering investigation. The script uses the credentials of pre-registered accounts to submit data. We make the data submission process natural so that it is not likely to raise an alert on the server. These Sybil users submit data as if they were in our POI (fictitious business), in a five m^2 office; we do not unleash all Sybil users at once, as it would be unrealistic to have more than a few people in a small room. We select a portion of the users and switch to a new subset of users every hour to make data submissions from the purported place look normal. Random submission rates create a different popular time graph for each day. For example, only 40% of the Sybil users submit data between 10:00-11:00 on Tuesdays, but 60% submit on Wednesdays. Users submit data only within the opening hours of the business. In summary, the script loads the credentials of the users, the refresh tokens, registered *client_id*’s, and email addresses in the setup phase. The script then obtains the necessary tokens for data submission. Before submitting data, each adversary crafts the *x-geo* headers. Finally, each user submits single location data to the Google Maps server every second. Script checks and refreshes tokens that are near to their expiration time.

Algorithm 1 shows the attack procedure for a specific POI target. Note that the algorithm is a basic version, meaning that one can scale the number of users or submission rate for the intended attack. First, the script takes the user set with the relevant credentials and targeted POI coordinates as input. It then checks and obtains the relevant tokens for each user and saves the expiration time (lines 2-8). Note that the token acquisition has a slight difference for iOS

and Android as shown in the Fig. 2. Android devices obtain only access tokens and submit data with them, whereas iOS devices additionally obtain submit tokens. Then, for each user, it first builds the *x-geo* header with the target location and current time and then submits the location data using the token it obtained previously (lines 11-12). Finally, if the script successfully submits the data and gets the "200 OK" code from the server, it proceeds with the next user. Otherwise, the script repeats the same procedure. To imitate the app behavior, the script also waits for 1 second for the subsequent submission of the same user.

Algorithm 1 Orchestration of Script-based Place Attacks

Input: User set with credentials: U , target coordinates: loc_{target}

```

1: for all  $s$  such that  $s \in U$  do
2:    $s_{token}, s_{expiry} \leftarrow obtain\_access\_token(s_{RT})$ 
3: end for
4:
5: while True do
6:   for all  $s$  such that  $s \in U$  do
7:     if  $s_{expiry} - t_{now} \leq ONE\_SECOND$  then
8:        $s_{token}, s_{expiry} \leftarrow obtain\_access\_token(s_{RT})$ 
9:     end if
10:    repeat
11:       $x_{geo} \leftarrow create\_x_{geo}(loc_{target})$ 
12:       $isSubmitted \leftarrow submit(s_{token}, x_{geo})$ 
13:    until  $\neg isSubmitted$ 
14:   end for
15:    $wait(1)$ 
16: end while

```

Each of the fake (Sybil) users encodes the *x-geo* headers to emulate user movement by calling *create_x_geo(loc)* function. Usage of latitude and longitude facilitates this process. We create an abstract rectangle as the targeted region. An attacker picks a random location in the created square to simulate the movement as persistent location submission without changes might raise the alarm on the server-side. We found Google Maps does not block these constant location submissions. The timestamp is set individually for each fake user, so each timestamp is distinct. We set the radius, role, and producer of the location data to the value we found in the captured packets. In other words, all generated *x-geo* headers will be unique. Google Maps does not accept identical consecutive *x-geo* headers. Further, attackers in our experiments change the 'safe' bytes randomly in the payload. This small change prevents the server from trivially linking the payloads.

We confirm our script-based place attacks by affecting live popular-times graphs. Only popular businesses with high user activity have the live crowdedness levels available [9]. These graphs changes at the beginning of each hour. Accordingly, we assume that live crowdedness is based on the crowdedness over the previous hour. Our fake business does not show the popular times graph because not every business receives this feature. We contacted Google Business support and received no answer though the server accepts our submitted data, we could not see a tangible effect. To responsibly obtain such evidence, we use a real-world business for experiments. After obtaining their consent on the experiment. We picked the

least popular hours to minimize the possible complications for the business. We use the credentials of the fake accounts we registered to both emulators and real mobile phones. We were physically present on the premises to verify the results. We started submitting data with ten distinct accounts and observed the popular-times graph change as shown in Fig. 3. The crowdedness levels increased even though the actual customer number decreased by two. When we increased the number of submissions per account, we could not see any increase in crowdedness. This is normal since Google only uses the data of the users who allowed their data to be stored historically [12].

The scripts are efficient in terms of computation, running Alg. 1 with ten accounts only use 5% of the CPU and 25 MB of memory. We use one IP address for the submissions. This is unlikely to raise any alarm because barring submissions with the same IP creates a compatibility nightmare. Such counter-measure, for example, may prevent devices that use access points to connect to the internet. On average, we submit 75,000 location data daily to our fake business for eight hours. The server accepts all submitted data. We verified that if the submission form has corrupted or wrong values, e.g., the wrong token or client id, the server rejects the submitted data.

We consider the scalability of the script-based place attacks by submitting large amounts of location information. There are no submission limits for users. In other words, users can make multiple submissions concurrently. Besides, users can use the same submit/access tokens for multiple submissions. This weakness allows Sybil users to appear in many places simultaneously, affecting the PS campaigns in a broad area. For this experiment, we replicate the Sybil users to have 700 'sources' of data submission simultaneously, i.e., 70 threads for each account submit data simultaneously. We use our fake business for the location of submissions. The server accepts all of the serially submitted data except for some rare cases around eight seconds. This experiment shows that an attacker can submit a vast volume of data without being evicted from the system. Even if Google detects such Sybil-based activity, it is hard to remove its effects. Evicting a user/blocking an account may be easy, but deleting the huge amount of data already contributed can be challenging since it is unclear when misbehavior started, more so given data are being aggregated.

6 EMULATOR-BASED ATTACKS

We explore how an attacker could affect business, public transportation, and street crowdedness information leveraging emulator-based Sybil users. The motivation, target, and victims can be the same as those for script-based place attacks. The difference is the means and method of the attack: the attacker uses smartphone emulators for submitting location data to change the crowdedness of the targeted road/business/public transportation instead of scripts. Emulators run instances of Google Maps simultaneously and submit incorrect data. The advantage from the attacker's viewpoint is that emulators are easier to configure, and use; the disadvantage is they are more costly to operate. Installing the app to the emulators and setting their GPS to the attack target can be easy for most people. The attacker needs to host the emulator in her local machines, which can be performance- and energy-heavy. Alternatively, the

attacker can host the emulators in the cloud, which can be expensive. All in all, emulator-based attacks can be a practical choice for the attackers if they have the necessary resources.

For our experiments, we use a workstation to run emulators locally. We use Genymotion [25] for the emulator software, but other tools can be used (excluding Android Studio because some of Google Maps features seem to not work in emulators). We use the free version of Genymotion, with limited functionality. The pro version of Genymotion can automate tasks better, e.g., automate and replay the GPS movement with a defined speed; or utilize GMTTool [26] to start emulators via command-line interfaces automatically. We establish that even the weakest attacker (without the Genymotion Pro) can be effective. A stronger attacker can scale these resources. We create helper scripts to automate the attack process. These scripts do not directly submit data, but help attackers orchestrate the attack.

6.1 Place Attacks

The attacker imitates an ordinary user, replacing physical devices with emulators. In other words, an attacker in a remote location with a computing platform tries to create an effect that would normally need multiple users with their physical devices actually present in the targeted area. The attacker first configures the emulators by installing the app, signing in with different accounts, and setting their location according to the targeted place. Within the targeted area, the attacker continuously changes the location of the emulators to create a fake movement. We randomly add some noise to the coordinates so that the location is different but inside the target place; submitting the same location data could alert the server and possibly allow it to link data by adversarial Sybil users. The runtime of this attack depends on the purpose of the attacker and can vary. For example, an attacker can run emulators for the entire working hour of the target place to create a constant crowd.

We use scripts to configure and control emulators to automate the attack process. The script takes attack target coordinates and total time to traverse the place as an input and it generates coordinates of the entire Sybil user trajectory intersecting the area of the target place; meaning that the Sybil user is present in the place itself. In addition, the script creates commands for configuring the emulators, such as selecting the emulator, enabling the GPS, and setting the device location. We then feed those commands to the emulators via the command line tool of Genymotion [27]. The advantage is that the attacker can automate the process of changing the emulator coordinates with prepared trajectories instead of manually entering them. Reproduction of the attack for any other targeted POI is more effortless in this way.

Our machine can host up to 9 emulators with 1 GB of memory simultaneously. We repeat the experiment in Sec. 5, based on scripts, to establish the attack can affect POI crowdedness histograms. We started 9 emulators, set their locations at 45 minutes, and stopped at the hour mark. We aimed at a slight increase of the live popular times of the graph. We stopped the emulators immediately once we saw a small effect like in Fig. 3. Thus, we verified that such emulator-based place attacks work if appropriately configured. Naturally, an attacker with more resources would only need to increase the

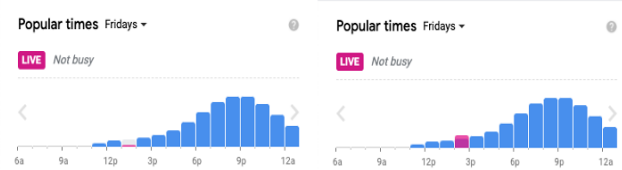


Figure 3: Popular times graph of a targeted business shown before and after the place attacks. We purposely only mildly influenced the crowdedness to avoid any inconvenience. An attack with a larger volume of submissions can be more pronounced or severe.

number of emulators and the effect of the attack would increase proportionally.

6.2 Public Transportation Attacks

The essence of the public transportation attacks is the attacker simulates the behavior similar to our example case described in the Sec. 2. One can set the locations manually or using a command-line shell [27]. We craft an attacker script that prepares configurations for the emulators and creates a route for emulated users. The scripts generate the necessary commands for attack automation, such as preparing Sybil user trajectories to create a fake movement. We also set locations and configurations manually via GUI to create movement for the emulators. In both cases, emulators can create a fake movement that misleads the that there is an actual user of a public transportation. When an emulator changes its location towards the expected public transportation route at the departure time, the app immediately sends a notification asking about the crowdedness level. As a result, the attacker does not have to wait for the planned trip route to reach the destination. Rather, every emulator obtains crowdedness forms within the first minute of the attack. This also shows that, if designed carefully, the attacker may not need to expend significant resources, running the emulation for a short time for each crowdedness form. We try to obtain crowdedness forms with a single account using many emulators simultaneously to see if we can scale our attacks. We log in to different emulator instances with the same account and aim to get forms for all emulators. We configure emulators as identically as possible. For example, we leave the IMEI number as 0 for all of them. Although we emulate the usage of different transportation lines simultaneously, each emulator obtains the crowdedness forms. We repeat targeting the same public transportation line, with the same setting, and again succeed in obtaining forms.

We repeat the experiments multiple times with different public transportation lines, at different times across a metropolitan area. On all occasions, the attacker obtains the crowdedness form and is thus allowed to submit data. Overall, PS-based public transportation live data anywhere in the city can be 'poisoned' with relatively small effort. This is a troubling sign on multiple fronts: The attacker does not even need multiple accounts. Google Maps does not check whether a device is 'real' or not. Our objective is to show that attackers can get these forms easily; the submission of incorrect data is trivial, but we refrain not to affect the application and benign users. In our experiments, we never fill or submit crowdedness forms to

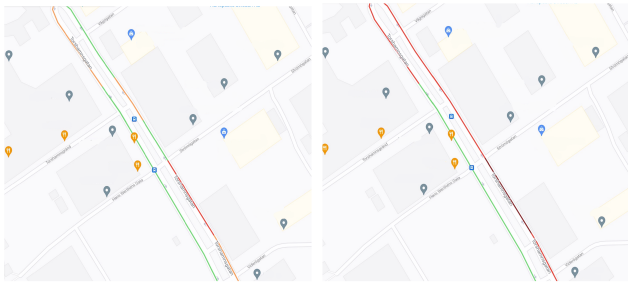


Figure 4: Illustrating the effect of the 12 emulated Sybil devices on a two-lane street. The road section turns from green to red after the slow-moving fake platoon passes.

not to pollute the collected data. We simply skip the crowdedness questions in case of form retrieval and close the app.

User Feedback Automation: The adversary could automate these attacks by auto-clicker applications [28, 29] found easily in the application marketplaces. An auto-clicker application allows attackers to configure actions, such as writing to search fields, clicking buttons, changing locations, and answering the crowdedness forms. With such tools, an attacker would not need to spend time guiding the attacks manually. Once designed, the auto-clicker configurations can be used for other emulators by duplicating the emulator image or distributing the configuration file. We use an auto-clicker app, specifically [28], to assist the attacks. Our auto-clicker configuration starts app, clicks the search button, enters the destination, selects the public transportation option, selects and starts the route. Simultaneously, we use another script to enable GPS, set the emulator's starting location, and create movement when the departure time comes. In conclusion, the attacker needs to open the auto-clicker software, load the configuration, and run the attack script on the host device for each emulator. The rest of the attack sequence is handled by the script and the auto-clicker, to obtain and submit forms automatically.

6.3 Traffic Attacks

We examine the attacks influencing traffic congestion levels using emulators which emulators mimic the behavior of legitimate users, e.g., smartphone of users in vehicles. Once again, we configure the emulators by installing the app. This time, we do not have any active account logged in to the app to show that this feature can be easier to attack. Each emulator is identical. We allocate 700 MB of memory and a single processor for each emulator. We also install and use an auto-clicker software [28] to automate the attack process. The auto-clicker opens the map, gets directions to the targeted place, configure the drive mode, and starts the route.

To move the emulators at a constant speed, we generate the coordinates of the devices before deploying the attack using a script. The script takes source and destination coordinates of the route and speed of the vehicles and outputs the commands to set these values for the emulators and create a movement. We feed these commands when Google Maps in the emulators start the route.

We use 12 emulators on three 450 meter long different street sections. Emulators move at a constant speed of 10 km/h. As the

command-line interface creates a slight delay when setting the GPS coordinates, thus the resultant emulated device speed is 1-2 km/h slower. We start the emulators at 5-second intervals to create a platoon or line of vehicles. As illustrated in Fig. 4, we started to see the effects after the first 250 meters, where the first section turned green to yellow. When the emulated platoon 'drives' for 800 meters, the corresponding road segment, all the roads up to that point turned to either red or dark red, meaning the slowest possible category. The remaining road sections turned from green to yellow to red when the platoon crossed the respective sections. We stopped the attacks after the fake platoon arrived end of the second road section to prevent further effects on the roads. We observed that the affected road sections remained 'slow' for approximately 10 minutes, even after the trial termination.

For the targeted road segment, we choose streets near our lab over a weekend evening. The area is populated by offices, so they are almost empty except for the work hours on weekdays. The street's historical traffic level is green (no traffic) for almost all weekend, but still we chose evenings to be extra careful. Although this sought to establish the attack worked, tried in a carefully selected location, basically isolated and with no actual traffic, to eliminate any risk for actual Google Maps users, there are significant ramifications. An attacker could produce large-scale artificial traffic jams in any city using emulator-based traffic attacks for long periods covering long or central, popular roads, affecting large numbers of users, and affecting Google Maps.

7 IMPLICATIONS AND COUNTERMEASURES

Implications: We showed that Sybil-based attacks can be debilitating and easy to mount, exactly because of the open and contributory nature of participatory sensing features in Google Maps. Combined with the automation of emulators and scripts, attackers could mount, scale, and steer the attacks quickly and easily. Altered knowledge for a target area or point of interest may be modest compared to the vast Google Maps world, but it can be highly detrimental to the individuals, the region, and the community it pertains to.

Google Maps is integrated into daily life thus attacks could have severe repercussions in the future if not addressed. Script-based attacks can alter the popularity and crowdedness of the places and transportation lines. An attacker could make a competitor restaurant look crowded on Friday nights to attract potential customers to her place. Another attacker could change the live crowdedness of one subway line to direct people to other lines to cause confusion and inconvenience. Emulator-based attacks could target public transportation and traffic congestion levels. An attacker could submit crowdedness forms to make a bus appear packed so that people may decide to use another, possibly longer or more expensive, way of transportation to avoid the crowded bus. Attackers could make central, popular roads look congested to force people to change routes, costing time and money. These kinds of Sybil attacks can come in a Cyberattack scenario conducted by countries against targeted countries' key cities. Cyberattacks can disrupt and cripple the specific vital areas of the city to create misinformation. As reliance on such PS-based information grows, in the future, without

appropriate countermeasures in place, large-scale misinformation attacks can disrupt important aspects of smart-city activities.

Countermeasures: We do not encounter any mechanisms to detect fake Sybil devices based on our experiments. We conjecture that Google Maps can detect and prevent most of our attacks relatively easily but not without tradeoffs. Once an attack is detected, Google Maps should block Sybil devices/sessions to prevent further harm. Next, we briefly discuss possible countermeasures.

Currently, refresh tokens allow script-based attacks. Refresh tokens are easy to obtain, have an unlimited lifetime, and can be used simultaneously. Server-side changes can address these weaknesses to detect Sybil devices and prevent them from harming the system. It is easy to capture the Google Maps traffic, thus allowing device owners to see messages, most importantly, refresh tokens. With the refresh tokens, attackers do not need any real device. They can utilize the scripts for data submission. Refresh tokens should not have an unlimited lifetime, thus making them harder to maintain. Instead of extracting them once, attackers would need to capture them continuously. Moreover, the server should not issue multiple, overlapping tokens. Overlapping tokens allow accounts to appear in multiple places and with multiple sessions simultaneously. Enforcing a single active token policy would eliminate Sybil users submitting data simultaneously. Still, at the same time, it can create compatibility problems for an app used by millions.

Sometimes legitimate users submit location data in entirely different places. The server can check recent data for any unusual submission. In case of detection of an anomaly, the server should discard the suspected data for the PS tasks without reducing other application functionality. This can happen when sensors are being calibrated or performed inaccurately (e.g., in a challenging propagation environment for GPS). Such data are faulty and should not be used in the PS tasks anyway. There are available data verification schemes in the literature that can help to solve this problem [30, 31], leveraging historically submitted data to establish a model and then test the incoming data in real-time. One can also detect the emulators by simple checks, such as inspecting IMEI, validating sensors, and detecting rooted devices [32]. Furthermore, when submitting crowdedness forms, Google's reCAPTCHA [33] can be used to prevent bots from automating the attacks.

Finally, given that an attacker can easily create many accounts for script-based attacks: Keeping a simple and open registration process may attract new users, but it also creates problems. Most people do not often create accounts for platforms of the Google caliber. So making the registration process heavier can be a reasonable cost for increasing security. However, harder registration may decrease participation and complicate user privacy. These defense recommendations hint at how to mitigate the attacks on the PS-based features of a popular LBS. They raise the bar for the attackers and make the system more secure as a first step towards thwarting all shown attacks.

8 RELATED WORK

There are PS applications aspiring to improve our quality of life for, among many proposals, healthcare [34, 35], environmental monitoring [36, 37] and urban sensing [38, 39] already with a significant user base. Security and privacy issues in participatory sensing have

received notable attention from the research community [40, 1, 41, 2, 42] but deployed applications with PS functionality and features have hardly adopted security mechanisms. As smart devices become increasingly popular and soon ubiquitous, it is only a matter of time that increased use of PS and data contributory systems leads to exploits, misbehaviors, and attacks. There are few works focused on the effects of Sybil attacks in the context of PS. [43, 44] focuses on attacks towards location-based such as Foursquare and Facebook places. A more recent work [45] outlines the investigation of attacks on different PS and LBS apps. However, their main focus is API requests and improper input validation vulnerabilities of the apps and how an attacker can exploit them to influence the system, whereas this work focuses on fake users within the system.

More closely related research investigated attacks on the traffic congestion levels of Google Maps, and Waze [46]. Even though it is based on the 2013 versions of the apps, it gives insights regarding the PS aspects, and we can still see the similarities in data encoding in the current version. In addition, [46] proposes a defense mechanism verifying the submitted location via zero-knowledge proofs. Ghost Riders [6] is perhaps the most closely related work to ours, creating Sybil, ghost riders in the popular app Waze. These ghost riders manipulate the system by creating fake events and traffic congestion levels. The proposed countermeasure is based on SybilRank [47] algorithm that ranks Sybil nodes based on their relation to the social network of the system. Similar prior work in [48] focuses on such Sybil drivers in Waze, too, creating fake traffic jams and impacting real-time navigation services. [49] investigates the fake businesses in Google Maps and discusses the ramifications by analyzing past search queries and how attackers bypass the security mechanisms for business verification. Finally, [32] aims at preventing the Sybil node registration to a PS system by checking whether devices are real or emulated, along with countermeasures relevant here. Comparing to the most relevant works [6, 48, 46], we focus on a broader range of PS functionality and demonstrate this on a different, popular service. We consider Google Maps popularity graphs, public transportation, and traffic data, whereas others focus on traffic events, congestion levels, and user tracking. This paper and [6, 48, 46] all focus on Sybil-based attacks, especially concerning traffic data, but the app, the PS features, and the attack specifics differ.

9 RESPONSIBLE DISCLOSURE AND CONDUCT

We disclosed our findings and attacks to Google in late April 2021. In June 2021, we supplemented the disclosure with a detailed description of the exact steps to reproduce the attacks. The disclosure [50] was done through Google Vulnerability Reward Program (VRP) [51]. The Google Bug Hunter Team evaluated our submission and accepted the vulnerability in July 2021. It was classified with priority level *P2* and severity level *S2* [52]. A bug 'bounty' was provided as a reward, together with an acknowledgment in the hall of fame [53]. Google is working on a fix, with work ongoing as of 30th of March 2022.

Our work sought to establish that the envisioned attacks, essentially, work while ensuring that our investigation did not affect any actual Google Maps user. We proceeded with each investigation

step with utmost care. First, we considered a self-defined POI without any significance to any actual user. Then, we sought the consent of a real-world business, picked an appropriate time (low popularity slot, with very few customers), and dimensioned the duration and Sybil-based data volume to a minimum, to observe a small change in the POI crowdedness histogram. We immediately stopped the experiment as soon as the observation was made. When considering the public transportation crowdedness, obtaining a questionnaire for a targeted line/route sufficed to establish the attack worked; we refrain from submitting the slightest response.

Regarding establishing that the attacks work for the PS-based traffic level information, we adhered to the same principle. First, we refrained from reenacting scenarios as those the most closely related work has already demonstrated; attack-induced long road segment congestion or road hazards [48], or suboptimal route selection due to the attacks [6]; or several-minute walk of 99 devices on the road pavement or across a bridge to be perceived as Google Maps road congestion [5]. First, we choose a short road segment to target, we emulate only 12 users, and we do not even attempt to demonstrate any effect such as a route change or road hazard; only a very ephemeral modest change on the reported congestion, and then immediately stop the experiment. Overall, the short-lived modest change would hardly affect return routing options even for artificial local queries.

Equally important, we select the location of the targeted road segment in an area and the time of the experiment at a time such that there is practically no activity. The road segment, in a tech office area outside the city center, has a low driving speed limit serving only local traffic, in a grid road city planning layout offering multiple options. It is not close to any important passage or complicated geography (e.g., bridge connecting islands) or traffic arteries (e.g., motorways, tunnels). Moreover, popular POIs in the region (e.g., commercial centers) can be accessed easily through other major roads. More so, the time, a weekend evening ensured that there would practically no one driving to and from the road segment. Based on long observations, activity drops drastically in the evenings, and it is practically non-existent during weekends, as people refrain from going to the office – more so on Saturday night. This scarcity of people on weekend evenings was further aggravated during these pandemic times, with even less human mobility in the area. Last but not least, to the best of our knowledge, no emergency service vehicle relies on Google Maps for navigation.

We also note that we were careful not to violate any Google policy [54] while creating and using accounts, notably not buying accounts nor creating them in an automated manner and not using them for any other purpose beyond the investigation for this research effort. Again, we established the feasibility of the attacks only, with a small number of accounts, without investigating the (potentially catastrophic) impact of the attacks. One could have considered investigating large-scale, large-volume real-world attacks, but this would unavoidably result in significant disruptions of the service and significant risks for users. Thus, a different methodology would have been necessary.

10 CONCLUSION

We presented Sybil-based attacks to reveal the vulnerabilities of the current popular PS-based LBS. We proposed different types of attacks targeting different PS features of Google Maps, a well-established and very popular LBS. The motivation of this work is to establish that such features can be targeted and exploited by automated, large-scale, large-volume attacks, impacting numerous users. This is more important as, in the future, more users and more activities in a smart city would depend on PS-based LBS; especially because these findings can carry to other applications and services. On the positive side, our investigation that identified attacks already hints on specific countermeasures that could be integrated and thwart Google Maps attacks. More broadly, given popular applications with PS features lack strong security, despite research proposing full-fledged architectures for PS security and privacy, we hope this work further motivates improvements and eventually trustworthy PS.

ACKNOWLEDGMENTS

This work was partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation, by the Swedish Foundation for Strategic Research (SSF) SURPRISE project, and the KAW Academy Fellowship Trustworthy IoT project

REFERENCES

- [1] Stylianos Gisdakis, Thanassis Giannetsos, and Panagiotis Papadimitratos. 2016. Security, privacy, and incentive provision for mobile crowd sensing systems. *IEEE Internet of Things Journal*, 3, 5, 839–853.
- [2] Emiliano De Cristofaro and Claudio Soriente. 2013. Extended capabilities for a privacy-enhanced participatory sensing infrastructure (pepsi). *IEEE Transactions on Information Forensics and Security*, 8, 12, 2021–2033.
- [3] Hongyu Jin and Panos Papadimitratos. 2019. Resilient privacy protection for location-based services through decentralization. *ACM Trans. Priv. Secur.*, 22, 4. ISSN: 2471-2566. DOI: 10.1145/3319401. <https://doi.org/10.1145/3319401>.
- [4] Ivica Šuran. [n. d.] Fake road closure - google maps community. [Online; accessed 11-February-202]. (). <https://support.google.com/maps/thread/137301768/fake-road-closure?hl=en>.
- [5] Simon Weckert. [n. d.] Google maps hacks. [Online; accessed 10-January-2021]. (). <http://www.simonweckert.com/googlemaphacks.html>.
- [6] Gang Wang, Bolun Wang, Tianyi Wang, Ana Nika, Haitao Zheng, and Ben Y Zhao. 2018. Ghost riders: sybil attacks on crowdsourced mobile mapping services. *IEEE/ACM transactions on networking*, 26, 3, 1123–1136.
- [7] Google. 2009. The bright side of sitting in traffic: crowdsourcing road congestion data. [Online; accessed 14-December-2019]. (2009). <https://googleblog.blogspot.com/2009/08/bright-side-of-sitting-in-traffic.html>.
- [8] Andrew J. Hawkins. 2019. Google will now tell you how crowded your bus or train is likely to be. [Online; accessed 17-April-2020]. (2019). <https://www.theverge.com/2019/>

- 6 / 27 / 18761187 / google - maps - transit - crowded - delays - predictions - train - bus - subway.
- [9] Google. [n. d.] Popular times, wait times, and visit duration. [Online; accessed 14-December-2019]. (). <https://support.google.com/business/answer/6263531?hl=en>.
 - [10] Johann Lau. [n. d.] Google maps 101: how ai helps predict traffic and determine routes. [Online; accessed 29-November-2021]. (). <https://blog.google/products/maps/google-maps-101-how-ai-helps-predict-traffic-and-determine-routes/>.
 - [11] Tim Dierks and Eric Rescorla. 2008. The transport layer security (tls) protocol version 1.2. (2008).
 - [12] Google. [n. d.] Google my business help - how can i get google "popular times" to appear underneath my business page? [Online; accessed 14-December-2019]. (). <https://support.google.com/business/thread/1774142?hl=en>.
 - [13] Quora. [n. d.] How can i create a google account without a phone number during the registration process? [Online; accessed 10-January-2021]. (). <https://www.quora.com/How-can-I-create-a-Google-account-without-a-phone-number-during-the-registration-process>.
 - [14] Cyclonis. [n. d.] How to create a gmail account without a phone number? [Online; accessed 17-December-2020]. (). <https://www.cyclonis.com/how-to-create-gmail-account-without-phone-number/>.
 - [15] Hottelecom. [n. d.] Hottelecom voip phone service. [Online; accessed 10-January-2021]. (). <https://hottelecom.biz/>.
 - [16] AppSally. [n. d.] Appsally buy gmail accounts. [Online; accessed 10-January-2021]. (). <https://www.appsally.com/products/gmail-accounts/>.
 - [17] XK72. [n. d.] Charles proxy. [Online; accessed 21-Nov-2020]. (). <https://www.charlesproxy.com/>.
 - [18] Jeremy Rowley. [n. d.] Stop certificate pinning. [Online; accessed 1-December-2021]. (). <https://www.digicert.com/blog/certificate-pinning-what-is-certificate-pinning>.
 - [19] Lucian Constantin. [n. d.] Security researchers lose faith in http public key pinning. [Online; accessed 1-December-2021]. (). <https://thenewstack.io/security-researchers-lose-faith-http-public-key-pinning/>.
 - [20] OAuth2. [n. d.] OAuth2 access tokens. [Online; accessed 10-January-2021]. (). <https://www.oauth.com/oauth2-servers/access-tokens/>.
 - [21] Google. [n. d.] Chromium open source code. [Online; accessed 17-December-2020]. (). <https://github.com/chromium/chromium>.
 - [22] Google. [n. d.] X-geo spec. [Online; accessed 17-December-2020]. (). <https://goto.google.com/xgeospec>.
 - [23] Google Developers. [n. d.] Protocol buffers. [Online; accessed 17-December-2020]. (). <https://developers.google.com/protocol-buffers>.
 - [24] Google. [n. d.] Chromium partner_location_descripton protobuf file. [Online; accessed 17-December-2020]. (). https://github.com/chromium/chromium/blob/master/chrome/android/java/src/org/chromium/chrome/browser/omnibox/geo/partner_location_descriptor.proto.
 - [25] Genymobile. [n. d.] Genymotion android emulator. [Online; accessed 17-December-2020]. (). <https://www.genymotion.com/>.
 - [26] Genymobile. [n. d.] Genymotion android emulator. [Online; accessed 19-October-2021]. (). https://docs.genymotion.com/desktop/latest/06_GMTool.html.
 - [27] Genymobile. [n. d.] Genymotion android emulator. [Online; accessed 19-October-2021]. (). https://docs.genymotion.com/desktop/latest/05_Genymotion_Shell.html.
 - [28] Y.C Studio. [n. d.] Click assistant - auto clicker : gesture recorder. [Online; accessed 26-November-2021]. (). <https://play.google.com/store/apps/details?id=com.rise.automatic.autoclicker.clicker>.
 - [29] ArloSoft. [n. d.] Macroddroid - device automation. [Online; accessed 17-December-2020]. (). <https://play.google.com/store/apps/details?id=com.arlosoft.macroddroid>.
 - [30] Stylianos Gisdakis, Thanassis Giannetsos, and Panos Papadimitratos. 2015. Shield: a data verification framework for participatory sensing systems. In *Proceedings of the 8th ACM Conference on Security & Privacy in Wireless and Mobile Networks*, 1–12.
 - [31] Dong Wang, Tarek Abdelzaher, Lance Kaplan, and Charu C Aggarwal. 2013. Recursive fact-finding: a streaming approach to truth estimation in crowdsourcing applications. In *2013 IEEE 33rd International Conference on Distributed Computing Systems*. IEEE, 530–539.
 - [32] Jatesada Borsub and Panos Papadimitratos. 2018. Hardened registration process for participatory sensing. In *Proceedings of the 11th ACM Conference on Security & Privacy in Wireless and Mobile Networks*. ACM, 281–282.
 - [33] Google. [n. d.] Recaptcha. [Online; accessed 17-December-2020]. (). <https://www.google.com/recaptcha/about/>.
 - [34] Hong Lu, Denise Frauendorfer, Mashfiqui Rabbi, Marianne Schmid Mast, Gokul T Chittaranjan, Andrew T Campbell, Daniel Gatica-Perez, and Tanzeem Choudhury. 2012. Stresssense: detecting stress in unconstrained acoustic environments using smartphones. In *Proceedings of the 2012 ACM conference on ubiquitous computing*, 351–360.
 - [35] Jason Ryder, Brent Longstaff, Sasank Reddy, and Deborah Estrin. 2009. Ambulation: a tool for monitoring mobility patterns over time using mobile phones. In *2009 International Conference on Computational Science and Engineering*. Volume 4. IEEE, 927–931.
 - [36] Michael Von Kaenel, Philipp Sommer, and Roger Wattenhofer. 2011. Ikarus: large-scale participatory sensing at high altitudes. In *Proceedings of the 12th Workshop on Mobile Computing Systems and Applications*, 63–68.
 - [37] Diego Mendez, Alfredo J Perez, Miguel A Labrador, and Juan Jose Marron. 2011. P-sense: a participatory sensing system for air pollution monitoring and control. In *2011 IEEE International Conference on Pervasive Computing and Communications Workshops (PERCOM Workshops)*. IEEE, 344–347.
 - [38] Gahng-Seop Ahn, Mirco Musolesi, Hong Lu, Reza Olfati-Saber, and Andrew T Campbell. 2010. Metrotrack: predictive tracking of mobile events using mobile phones. In *International Conference on Distributed Computing in Sensor Systems*. Springer, 230–243.

- [39] Daniel Peebles, Hong Lu, Nicholas D Lane, Tanzeem Choudhury, and Andrew T Campbell. 2010. Community-guided learning: exploiting mobile sensor users to model human behavior. In *AAAI*. Volume 10. Citeseer, 1600–1606.
- [40] Ioannis Boutsis and Vana Kalogeraki. 2013. Privacy preservation for participatory sensing data. In *2013 IEEE International Conference on Pervasive Computing and Communications (PerCom)*. IEEE, 103–113.
- [41] Stylianos Gisdakis, Thanassis Giannetsos, and Panos Papadimitratos. 2014. Sppear: security & privacy-preserving architecture for participatory-sensing applications. In *Proceedings of the 2014 ACM conference on Security and privacy in wireless & mobile networks*, 39–50.
- [42] Jianbing Ni, Kuan Zhang, Qi Xia, Xiaodong Lin, and Xuemin Sherman Shen. 2020. Enabling strong privacy preservation and accurate task allocation for mobile crowdsensing. *IEEE Transactions on Mobile Computing*, 19, 6, 1317–1331. doi: 10.1109/TMC.2019.2908638.
- [43] Bogdan Carbutar and Rahul Potharaju. 2012. You unlocked the mt. everest badge on foursquare! countering location fraud in geosocial networks. In *2012 IEEE 9th International Conference on Mobile Ad-Hoc and Sensor Systems (MASS 2012)*. IEEE, 182–190.
- [44] Iasonas Polakis, Stamatis Volanis, Elias Athanasopoulos, and Evangelos P Markatos. 2013. The man who was there: validating check-ins in location-based services. In *Proceedings of the 29th Annual Computer Security Applications Conference*, 19–28.
- [45] Sojhal Ismail Khan, Dominika C Woszczyk, Chengzeng You, Soteris Demetriou, and Muhammad Naveed. 2021. Characterizing improper input validation vulnerabilities of mobile crowdsourcing services. In *Annual Computer Security Applications Conference*, 944–956.
- [46] Tobias Jeske. 2013. Floating car data from smartphones: what google and waze know about you and how hackers can control traffic. *Proc. of the BlackHat Europe*, 1–12.
- [47] Qiang Cao, Michael Sirivianos, Xiaowei Yang, and Tiago Pregueiro. 2012. Aiding the detection of fake accounts in large scale social online services. In *Proceedings of the 9th USENIX conference on Networked Systems Design and Implementation*. USENIX Association, 15–15.
- [48] Meital Ben Sinai, Nimrod Partush, Shir Yadid, and Eran Yahav. 2014. Exploiting social navigation. *arXiv preprint arXiv:1410.0151*.
- [49] Danny Yuxing Huang, Doug Grundman, Kurt Thomas, Abhishek Kumar, Elie Bursztein, Kirill Levchenko, and Alex C. Snoeren. 2017. Pinning down abuse on google maps. In *Proceedings of the 26th International Conference on World Wide Web (WWW '17)*. International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 1471–1479. ISBN: 9781450349130. doi: 10.1145/3038912.3052590. <https://doi.org/10.1145/3038912.3052590>.
- [50] Google. [n. d.] How google handles security vulnerabilities. [Online; accessed 19-Nov-2021]. (). <https://about.google/appsecurity/>.
- [51] Google. [n. d.] How google handles security vulnerabilities. [Online; accessed 19-Nov-2021]. (). <https://bughunters.google.com/about/rules/6625378258649088>.
- [52] Google. [n. d.] Issues | google issue tracker | google developers. [Online; accessed 1-December-2021]. (). <https://developers.google.com/issue-tracker/concepts/issues>.
- [53] Google. [n. d.] Google Bug Hunters - Cihan Eryonucu & Panos Papadimitratos. [Online; accessed 1-April-2022]. (). <https://bughunters.google.com/profile/d6aee20a-de54-44a9-a37e-fff7ee4b0675>.
- [54] Google. [n. d.] Gmail program policies. [Online; accessed 1-April-2022]. (). <https://www.google.com/gmail/about/policy/>.